1.1 Introduction to package dgeom

Note: This package does not contain support for differential forms. The 'cartan' package in core Maxima provides some basic support for forms. This package should be considered an add-on to the 'ctensor' package in core Maxima, providing further functionality for some tensor calculations, as well as some standard differential geometry tools.

The `dgeom` package provides some basic functions useful for exploring metrics and manifolds. In particular coordinate transformations that allow one to define embedded manifolds in flat space. The package also provides more general tools for computing metrics after a coordinate transformation. This package was designed to be compatible with the `ctensor` package included with core Maxima.

Major capabilities of `dgeom` include:

- The `dgeom` package computes the frame field connections for an embedded surface using frames defined by the attitude matrix. See the function `dg_ffc()`.
- Computation a new metric from an old metric where the old metric is not δ (identity) or η (Minkowski). This is especially useful for stringing together several coordinate transformations when studying a metric.
- Computation of the Killing vector equations from the metric using `dg_kill()`.

The basic use of the package begins by defining the coordinates in the variables `cords_in`, `cords_tr`, and `cords_ot`. This can be done manually or by using the built-in function `dg_cords`.

In a typical session, one should first call `dg_cords(all)` to see the available coordinate systems and then choose one, or set up the coordinates by hand. Second, call `dg_metric()` to compute the metric in the new coordinates. `dgeom` can calculate the divergence, gradient, curl (`d=3` only), and Laplacian in any coordinate system you can define. It also defines them as functions you can then call if you want to use them in a computation. These computations are available in the functions `dg_diverg()`, `dg_grad()`, `dg_curl()`, and `dg_laplac()`.

Note: It helps to simplify the resulting formulas by restricting coordinate ranges. For example, if studying the sphere one should use `assume(theta>0,theta<%pi)` and `assume(r>0)`. These are set automatically in the package, but for coordinate systems input by hand one should consider setting the ranges manually.

The `dgeom` package is compatible with the `ctensor` package. Coordinates defined in `dgeom` can be used for computations in the `ctensor` package such as the Christoffel symbols, the Riemann tensor and so forth. See the function `set_ctensor_vars` for more information.

version: 0.3

Eric Majzoub, Oct 2026

1.2 Functions and Variables for dgeom

`cord_in` [Variable]

Input coordinates are given as a list in the variable `cord_in`.

`cord_ot` [Variable]

Output coordinates are given as a list in the variable `cord_ot`.

`cord_tr` [Variable]

Coordinate transformations are given as a list in the variable `cord_tr`. `cord_tr:[r*sin(theta)*cos(phi),r*sin(theta)*sin(phi),r*cos(theta)]`.
This implies, for example, that if you use `cords_in=[x,y,z]`, then `x=r*sin(theta)*cos(phi)`, `y=r*sin(theta)*sin(phi)`, and `z=r*cos(theta)`.

If you want to use the `cnvrt2` option in `dg_metric`, then the use may specify the equations themselves in `cords_tr`, for example, transforming from standard Schwarzschild coordinates to tortoise coordinates one specifies `cords_tr:[t,R=r+2*M*G*log(r/(2*M*G)-1),theta,phi]`. See examples below that show this explicitly.

`dg_minkowski` [Variable]

The variable `dg_minkowski` (default value: 1) determines whether the metric in the output coordinates `cords_ot` is computed with a Minkowski signature when running `dg_metric(init)`. This variable is set automatically for the predefined coordinate transformations 'rindler'. If defining coordinates in Minkowski signature without using the function `dg_cords`, the timelike coordinate must be the first element in the list for both `cords_in` and `cords_ot`.

Some of the predefined coordinate systems do not need to have `dg_minkowski` set. These are the coordinate systems for which the metric is predefined: schwarzschild, kerr, eddfink, kruskal, and any other coordinate choice that shows the message indicating that `dg_metric(init)` need not be run.

`dg_cords` (*coordinate system*) [Function]

The function `dg_cords` takes a predefined coordinate system name and sets the variables `cords_in`, `cords_tr`, `cords_ot`. `cords_in` are the input coordinates, `cords_tr` are the transformation functions to the new or output coordinates, and `cords_ot` are the output coordinates. To see the available predefined coordinate systems use `dg_cords(all)`. Issuing `dg_cords(show)` will display the coordinate variables.

Some choices of coordinates have predefined metrics and these do not require running `dg_metric(init)`.

```
(%i1) dg_cords(rindler);
(%o1)                                done
(%i2) show_cords();
                               cords_in = [t, x]

                 cords_tr = [sinh(omega) rho, cosh(omega) rho]

                             cords_ot = [omega, rho]

                              dg_minkowski = - 1

(%o2)                                done
(%i3) dg_metric(init);
                                        2         2
                           ds2_in = del (x) - del (t)

                                    [      2    ]
                                g = [ - rho   0 ]
                                    [           ]
                                    [   0     1 ]

(%o3)                                done
```

`show_cords ( )` [Function]

The function `show_cords` takes no arguments. It displays the variables `cord_in`, `cord_tr`, `cord_ot`, and `dg_minkowski`.

`dg_derivs ( )` [Function]

This function takes no arguments, but the input and output coordinates and the transformation functions must be defined beforehand for this function to work. See the function [dg_cords], page 2. The function `dg_derivs` computes the partial derivative functions of the output coordinates in terms of the input coordinates using the transformation functions. For example, for input coordinates `[x,y,z]`, and output coordinates `[r,phi,z]`, and using the predefined cylindrical coordinate transformation `xyz_to_cyl`, one obtains the partial derivatives with respect to *(x,y,z)* in terms of the partial derivatives with respect to the variables *(r,phi,z)*.

The resulting partial derivative functions are named according to the input variable names, in the following notation. The partial derivative with respect to x is denoted `d_dx`, and likewise for the other input coordinates. The functions are defined with argument names generated by `gensym`.

```
(%i1) dg_cords(xyz_to_cyl);
(%o1)                                done
(%i2) dg_derivs();
(%o2)                                done
(%i3) fundef(d_dx);
                                 dF              dF
(%o3)              d_dx(F) := ---- sin(phi) + -- cos(phi)
                              dphi            dr
```

dg_jacobian ([*n*]) [Function]

Computes the Jacobian of the coordinate transformation. The Jacaobian is assigned to the matrix `J` and the inverse is assigned to the matrix `Jinv` The optional integer argument may be 0 (show no output), 1 (show `J`), 2 (show `Jinv`). The format for `cords_tr` must be a set of expressions in terms of `cords_ot`. For example:

```
cords_in = [x, y, z]
cords_tr = [r*cos(phi)*sin(theta),r*sin(phi)*sin(theta),r*cos(theta)]
cords_ot = [r, theta, phi]
```

dg_metric ([*init,cnvrt,cnvrt2,cnvrt3,cnvrt4*]) [Function]

This function takes as argument *init* or *cnvrt*. The input and output coordinates and the transformation functions must be defined beforehand for this function to work. See the function [dg_cords], page 2. The function `dg_metric` computes the metric in terms of the output coordinates `cords_ot`. If `dg_metric` is called with the argument *init* then the function assumes the input metric is diagonal with unit entries, although `dg_minkowski` may be set to -1 for the time component.

If calling `dg_metric` with the argument *init*, the initial metric is assumed to be flat, but maybe Minkowski, in the initial coordinates *cords_in*. For example, this input option may be used for the following *x,y,x* to *r,theta,phi* coordinates. These *cords_tr* imply that *x=r*cos(phi)*cos(theta)*, and so on.

```
cords_in: [x,y,z]
cords_tr: [r*cos(phi)*cos(theta),r*cos(phi)*sin(theta),r*sin(phi)]
cords_ot: [r,theta,phi]
```

If `dg_metric` is called with the argument *cnvrt*, then the function assumes there is an initial (non-trivial) metric given in the matrix `lg_in` with the ordering of the entries in accordance with the ordering of the variables in `cords_in`. The value of `dg_minkowski` is ignored for this case. `dg_metric` returns the line element ds^2 in the variable `ds2`, and the metric in the matrix `g`. An example set of coordinates is below. Again, these *cords_tr* imply that *x=r*cos(phi)*cos(theta)*, and so on.

```
cords_in: [x,y,z]
cords_tr: [r*cos(phi)*cos(theta),r*cos(phi)*sin(theta),r*sin(phi)]
cords_ot: [r,theta,phi]
```

If dg_metric is called with the argument *cnvrt2*, then the everything is the same as for argument *cnvrt* except that the variable *cords_tr* may contain the *equations* relating the 'in' and out 'ot' coordinates. An example set of coordinates is:

```
cords_in: [x,y,z]
cords_tr: [x=r*cos(phi)*cos(theta),y=r*cos(phi)*sin(theta),z=r*sin(phi)]

cords_ot: [r,theta,phi]
```

If dg_metric is called with the argument *cnvrt3*, then the everything is the same as for argument *cnvrt*. Example coordinate inputs are the same as for *cnvrt* above. This option attempts conversion using the standard equation for conversion of coordinates of a second rank tensor.

If dg_metric is called with the argument *cnvrt4*, then the variable *cords_tr* may contain equations relating the differentials of input and output coordinates. This works similarly to option *cnvrt2*. An example set of coordinates is:

```
cords_in: [t,r,theta,phi]
cords_tr: [del(p)=del(t)+r/(r-2*M*G)*del(r),r,theta,phi]
cords_ot: [p,r,theta,phi]
```

Notes: The options *cnvrt*, *cnvrt2*, *cnvrt3*, and *cnvrt4* use different methods to try constructing the transformed metric. This sometimes fails because the coordinates cannot be inverted. If none of the methods works please send me a message.

The following example converts from *x,y,z* coordinates to spherical coordinates.

```
(%i1) dg_cords(xyz_to_spher);
(%o1)                                done
(%i2) dg_metric(init);
                                 2         2         2
                    ds2_in = del (z) + del (y) + del (x)

                           [ 1  0         0        ]
                           [                       ]
                           [     2                 ]
                       g = [ 0  r         0        ]
                           [                       ]
                           [          2    2       ]
                           [ 0  0    r  sin (theta) ]

(%o2)                                done
```

The following example illustrates the conversion of the Rindler metric to new coordinates that mimic the Schwarzschild metric using all three algorithms in dg_metric.

Using dg_metric(cnvrt)

```
(%i1) cords_in:[omega,rho];
(%o1)                              [omega, rho]
(%i2) cords_ot:[t,xi];
(%o2)                                 [t, xi]
(%i3) cords_tr:[t/2,2*sqrt(xi)];
                                      t
(%o3)                               [-, 2 sqrt(xi)]
                                      2
(%i4) lg_in:matrix([-rho^2,0],[0,1]);
                                      [      2    ]
(%o4)                                 [ - rho   0 ]
                                      [           ]
                                      [   0     1 ]
(%i5) dg_metric(cnvrt);
                                      2           2    2
                        ds2_in = del (rho) - rho  del (omega)

                                        [ - xi  0  ]
                                        [          ]
                                    g = [       1  ]
                                        [  0    -- ]
                                        [       xi ]

(%o5)                                    done
```

Using `dg_metric(cnvrt2)`

```
(%i1) cords_in:[omega,rho];
(%o1)                              [omega, rho]
(%i2) cords_ot:[t,xi];
(%o2)                                 [t, xi]
(%i3) cords_tr:[omega=t/2,rho=2*sqrt(xi)];
                                       t
(%o3)                    [omega = -, rho = 2 sqrt(xi)]
                                       2
(%i4) lg_in:matrix([-rho^2,0],[0,1]);
                                   [      2    ]
(%o4)                              [ - rho   0 ]
                                   [           ]
                                   [   0     1 ]
(%i5) dg_metric(cnvrt2);
                                  2           2    2
                      ds2_in = del (rho) - rho  del (omega)

                                     [ - xi  0  ]
                                     [          ]
                                 g = [       1  ]
                                     [  0    -- ]
                                     [       xi ]

(%o5)                                  done
```

Using `dg_metric(cnvrt3)`

```
(%i1) cords_in:[omega,rho];
(%o1)                              [omega, rho]
(%i2) cords_ot:[t,xi];
(%o2)                                 [t, xi]
(%i3) cords_tr:[t/2,2*sqrt(xi)];
                                    t
(%o3)                              [-, 2 sqrt(xi)]
                                    2
(%i4) lg_in:matrix([-rho^2,0],[0,1]);
                                    [      2    ]
(%o4)                               [ - rho   0 ]
                                    [           ]
                                    [   0     1 ]
(%i5) dg_metric(cnvrt3);
                                    2           2    2
                     ds2_in = del (rho) - rho  del (omega)

Failure inverting coordinates. Trying serial solutions.
                                      [ - xi  0  ]
                                      [          ]
                                  g = [       1  ]
                                      [  0    -- ]
                                      [       xi ]

(%o5)                                   done
```

Using `dg_metric(cnvrt4)`. This example is Schwarzschild to Eddington-Finkelstein coordinates.

```
(%i1) dg_cords(schwarzschild)$
         Do NOT run dg_metric(init), the metric is already defined.

(%i2) lg_in:lg$
(%i3) cords_tr:[del(p)=del(t)+r/(r-2*M*G)*del(r),r,theta,phi];
                                    r del(r)
(%o3)            [del(p) = del(t) + ---------, r, theta, phi]
                                    r - 2 G M
(%i4) cords_ot:[p,r,theta,phi]$
(%i5) dg_metric(cnvrt4);
                                          2           2
          2    2          (r - 2 G M) del (t)   r del (r)
ds2_in = r  del (theta) - ------------------- + ---------
                                    r            r - 2 G M
                                                        2    2           2
                                                   + r  sin (theta) del (phi)

                     [   r - 2 G M                        ]
                     [ - ---------  1  0         0        ]
                     [       r                            ]
                     [                                    ]
                     [      1       0  0         0        ]
                g =  [                                    ]
                     [                  2                 ]
                     [      0       0  r         0        ]
                     [                                    ]
                     [                       2    2       ]
                     [      0       0  0    r  sin (theta) ]

(%o5)                                done
```

The following example transformation leaves the metric with off-diagonal components.

```
(%i1) lg_in : matrix([-1,0,0,0],[0,1,0,0],[0,0,1,0],[0,0,0,1])$
(%i2) cords_in : [T,X,Y,Z]$
(%i3) cords_tr : [t+r,r*sin(theta)*cos(phi),r*sin(theta)*sin(phi),r*cos(theta)]$
(%i4) cords_ot : [t,r,theta,phi]$
(%i5) dg_metric(cnvrt);
                              2         2         2         2
                  ds2_in = del (Z) + del (Y) + del (X) - del (T)

                           [ - 1  - 1  0         0        ]
                           [                              ]
                           [ - 1   0   0         0        ]
                           [                              ]
                       g = [            2                 ]
                           [  0    0   r         0        ]
                           [                              ]
                           [                2    2        ]
                           [  0    0   0   r  sin (theta) ]

(%o5)                                done
```

The following example, using `cnvrt2`, converts from standard Schwarzschild coordinates to radial tortoise coordinates. Note that Schwarzschild has a predefined metric, and it is not necessary to run `dg_metric(init)`.

```
(%i1) dg_cords(schwarzschild);
         Do NOT run dg_metric(init), the metric is already defined.

(%o1)                                done
(%i2) ds2;
                                                  2
       2    2           2 G M         2       del (r)
(%o2) r  del (theta) + (----- - 1) del (t) + ---------
                         r                       2 G M
                                              1 - -----
                                                    r
                                                         2    2           2
                                                      + r  sin (theta) del (phi)
(%i3) cords_in;
(%o3)                          [t, r, theta, phi]
(%i4) cords_tr:[t,R=r+2*M*G*log(r/(2*M*G)-1),theta,phi];
                                        r
(%o4)             [t, R = 2 G M log(----- - 1) + r, theta, phi]
                                      2 G M
(%i5) cords_ot:[t,R,theta,phi];
(%o5)                          [t, R, theta, phi]
(%i6) lg_in:ds2_to_lg(ds2,cords_in);
                  [   r - 2 G M                                ]
                  [ - ---------      0      0         0        ]
                  [       r                                    ]
                  [                                            ]
                  [                  r                         ]
                  [      0       ---------  0         0        ]
(%o6)             [              r - 2 G M                     ]
                  [                                            ]
                  [                           2                ]
                  [      0           0      r         0        ]
                  [                                            ]
                  [                                2    2      ]
                  [      0           0      0   r  sin (theta) ]
(%i7) dg_metric(cnvrt2);
                                        2           2
            2    2          (r - 2 G M) del (t)   r del (r)
ds2_in = r  del (theta) - ------------------- + ---------
                                     r            r - 2 G M
                                                         2    2           2
                                                      + r  sin (theta) del (phi)

Can't solve for coordinate r uniquely. Not substituting.
                    [   r - 2 G M                                ]
                    [ - ---------      0      0         0        ]
                    [       r                                    ]
                    [                                            ]
                    [               r - 2 G M                    ]
                    [      0        ---------  0         0        ]
               g = [                   r                         ]
                    [                                            ]
                    [                            2               ]
```

`ds2_to_lg` (`ds`^{2} `line element`,`cords_in`) [Function]

This function takes a line element in the form output by `dg_metric`, and a list of coordinates and returns a metric in the matrix `lg`. The second argument can be either a list of coordinates or the list `cords_in` which is appropriate for doing coordinate conversion of the metric.

```
(%i1) cords_in:[t,r,theta,phi]$
(%i2) f:1-2*m/r;
                                    2 m
(%o2)                           1 - ---
                                    r
(%i3) dom2:del(theta)^2+sin(theta)^2*del(phi)^2;
                          2            2          2
(%o3)                del (theta) + sin (theta) del (phi)
(%i4) ds2: -f*del(t)^2+del(r)^2/f+r^2*dom2;
                                                                      2
       2     2             2           2          2 m         2      del (r)
(%o4) r  (del (theta) + sin (theta) del (phi)) + (--- - 1) del (t) + -------
                                                   r                   2 m
                                                                   1 - ---
                                                                       r
(%i5) ds2_to_lg(ds2,cords_in);
                  [   r - 2 m                              ]
                  [ - -------     0     0         0        ]
                  [      r                                 ]
                  [                                        ]
                  [              r                         ]
                  [     0      -------  0         0        ]
(%o5)             [            r - 2 m                     ]
                  [                                        ]
                  [                      2                 ]
                  [     0         0     r         0        ]
                  [                                        ]
                  [                           2    2       ]
                  [     0         0     0   r  sin (theta) ]
```

1.3 Interfacing with the CTENSOR package of Maxima

`get_ctensor_vars` ( ) [Function]

This command is used to set the `dgeom` coordinate variables from those defined in the `ctensor` package. This command sets the `dgeom` coordinate variable `cord_ot` to the ctensor variable `ct_cords`. Then sets the lower indexed metric `lg` to the metric `g`. The `ctensor` package must be loaded and the metric computed for this command to work.

`set_ctensor_vars` ([*init*,*cnvrt*]) [Function]

This function takes as argument *init* or *cnvrt*. The input and output coordinates and the transformation functions must be defined beforehand for this function to work.

See the function [dg_cords], page 2. See [dg_metric], page 4, for a description of the options *init* and *cnvrt*. This command is used to set up the calculation of the Christoffel symbols using the `ctensor` package using the coordinates and transformation functions from the `dgeom` package. The Christoffel symbols are the connections in the coordinate frame, as opposed to the frame-field connection coefficients computed using the function `dg_ffc`. The function `set_ctensor_vars` sets the following `ctensor` variables: `cframe_flage:false`, `dim`, and calls `ct_coordsys()`. The `ctensor` package must be loaded for this command to work.

Example: Compute the Christoffel symbols for flat space in polar coordinates.

```
(%i1) load(ctensor)$
(%i2) dg_cords(xy_to_polar);
(%o2)                                done
(%i3) set_ctensor_vars(init);
                                      2         2
                          ds2_in = del (y) + del (x)

                                      [ 1  0  ]
                                  g = [       ]
                                      [     2 ]
                                      [ 0  r  ]

                ctlist = [r cos(theta), r sin(theta), [r, theta]]

(%o3)                                done
(%i4) christof(mcs);
                                      2      1
(%t4)                               mcs     = -
                                      1, 2   r

                                     1
(%t5)                               mcs     = - r
                                        2, 2

(%o5)                                done
```

1.4 Div, Grad, Curl and all that

The following section contains functions for generating the formulas for the laplacian, gradient, curl, and divergence, for arbitrary coordinate systems. There are two options for output of the formulas. The first is that output is in the physical basis, this is the basis for formulas in most textbooks. The second is formula output in the coordinate basis. This is the basis most used in general relativity and differential geometry. See AI for a description.

`dg_grad ([1,2])` [Function]

This function takes one argument. The input and output coordinates and the transformation functions must be defined beforehand for this function to work. See the

function [dg_cords], page 2. The function `dg_grad` returns an expression for the gradient of a function `F` in the output coordinates `cord_ot`.

If the argument to `dg_grad` is `1`, then the gradient is computed in the physical basis (found in most intro textbooks). If the argument is `2` the gradient is computed in the coordinate basis.

```
(%i1) dg_cords(xyz_to_spher);
(%o1)                                done
(%i2) dg_grad(1);
                                  2         2         2
                     ds2_in = del (z) + del (y) + del (x)

                              [ 1  0         0        ]
                              [                       ]
                              [     2                 ]
                          g = [ 0  r         0        ]
                              [                       ]
                              [          2    2       ]
                              [ 0  0   r  sin (theta) ]

gradient in physical basis
                               dF             dF
                              ---- e_phi     ------ e_theta
                              dphi           dtheta           dF
(%o2)          dg_gra(F) := ------------ + -------------- + -- e_r
                            r sin(theta)         r          dr
```

`dg_curl ([1,2])` [Function]

This function takes one argument. The input and output coordinates and the transformation functions must be defined beforehand for this function to work. See the function [dg_cords], page 2. The curl function is limited to `d=3` dimensions. If the argument to `dg_curl` is `1`, then the curl is computed in the physical basis (found in most intro textbooks). If the argument is `2` the curl is computed in the coordinate basis using the metric tensor.

The function `dg_curl` returns a function `dg_cur(W)` that takes vector argument `W` and returns the curl in terms of the output coordinates `cord_ot`.

```
(%i1) dg_cords(cart3d);
(%o1)                                done
(%i2) dg_curl(1);
                                 2         2         2
                      ds2_in = del (z) + del (y) + del (x)

                                 [ 1  0  0 ]
                                 [         ]
                              g = [ 0  1  0 ]
                                 [         ]
                                 [ 0  0  1 ]

curl in physical basis
                                  dW_z   dW_y  dW_x   dW_z  dW_y   dW_x
(%o2)  dg_cur(W_x, W_y, W_z) := [---- - ----, ---- - ----, ---- - ----]
                                   dy     dz    dz     dx    dx     dy
(%i3) dg_cords(xyz_to_cyl);
(%o3)                                done
(%i4) dg_curl(1);
                                 2         2         2
                      ds2_in = del (z) + del (y) + del (x)

                                   [ 1  0   0 ]
                                   [          ]
                                   [     2    ]
                                g = [    r     ]
                                   [ 0  r   0 ]
                                   [          ]
                                   [ 0  0   1 ]

curl in physical basis
                                   dW_z
                                   ----
                                   dphi   dW_phi  dW_r   dW_z
(%o4) dg_cur(W_r, W_phi, W_z) := [---- - ------, ---- - ----,
                                    r       dz     dz     dr
                                                          dW_r
                                                          ----
                                                          dphi   W_phi   dW_phi
                                                        - ---- + ----- + ------]
                                                          r       r       dr
```

dg_diverg ([1,2]) [Function]

This function takes one argument. The input and output coordinates and the transformation functions must be defined beforehand for this function to work. See the function [dg_cords], page 2. If the argument to `dg_diverg` is `1`, then the divergence is computed in the physical basis (found in most intro textbooks). If the argument is `2` the divergence is computed in the coordinate basis using the metric tensor.

The function `dg_diverg` returns a function `dg_div(W)` that takes argument `W` and returns the divergence in terms of the output coordinates `cord_ot`. The function `dg_metric` must be called before calling `dg_diverg`.

```
(%i1) dg_cords(xyz_to_cyl);
(%o1)                                done
(%i2) dg_metric(init);
                                  2         2         2
                     ds2_in = del (z) + del (y) + del (x)

                                    [ 1  0   0 ]
                                    [          ]
                                    [     2    ]
                                g = [    r     ]
                                    [ 0  r   0 ]
                                    [          ]
                                    [ 0  0   1 ]

(%o2)                                done
(%i3) dg_diverg(1);
divergence in physical basis
                                                 dW_phi
                                                 ------
                                            W_r    dphi    dW_z   dW_r
(%o3)        dg_div(W_r, W_phi, W_z) := --- + ------ + ---- + ----
                                             r      r       dz     dr
```

`dg_laplac ( )` [Function]

This function takes no arguments. The input and output coordinates and the transformation functions must be defined beforehand for this function to work. See the function [dg_cords], page 2.

The function `dg_laplac` returns a function `dg_lap(W)` that takes arument `W` and returns the Laplacian in terms of the output coordinates `cord_ot`. The function `dg_metric` must be called before calling `dg_laplac`.

```
(%i1) dg_cords(xyz_to_cyl);
(%o1)                                done
(%i2) dg_metric(init);
                                  2         2         2
                      ds2_in = del (z) + del (y) + del (x)

                                   [ 1  0   0 ]
                                   [          ]
                                   [     2    ]
                               g = [    r     ]
                                   [ 0  r   0 ]
                                   [          ]
                                   [ 0  0   1 ]

(%o2)                                done
(%i3) dg_laplac();
                                         2
                                        d F
                                   dF   -----
                                   --       2    2     2
                                   dr   dphi    d F   d F
(%o3)                dg_lap(F) := -- + ----- + --- + ---
                                   r      2       2     2
                                          r      dz    dr
```

1.5 Frame field calculations, and Killing vectors

The following section contains the commands for computing the frame field connections, and the killing vectors. The command set_ctensor_vars also allows one to set up variables for working with the ctensor package.

`dg_ffc` ([*constraint equations*]) [Function]

Compute the frame field connections. The input and output coordinates and the transformation functions must be defined beforehand for this function to work. See the function [dg_cords], page 2.

The function `dg_ffc` computes the attitude matrix `A`, the connection coefficients in the matrix `Omega`, and the dual 1-forms in the array `thi`. Any constraint equations given are imposed after calculating the attitude matrix.

The structural equations are computed by first computing the matrices `TT[i,j]: thi[i] . thi[j]`, and `DD[i,j]: diff(Omega[i,j])`, and then computing `KK[i,j]` by solving *domega(i,j) = K th(i) * thi(j).*

```
(%i1) dg_cords(xyz_to_cyl);
(%o1)                                done
(%i2) dg_ffc([r = 1]);
(%o2)                                done
(%i3) A;
                           [  cos(phi)   sin(phi)  0 ]
                           [                       ]
(%o3)                      [ - sin(phi)  cos(phi)  0 ]
                           [                       ]
                           [     0          0      1 ]
(%i4) Omega;
                           [     0       del(phi)  0 ]
                           [                       ]
(%o4)                      [ - del(phi)     0      0 ]
                           [                       ]
                           [     0          0      0 ]
(%i5) thi;
                                 [   del(r)   ]
                                 [            ]
(%o5)                            [ r del(phi) ]
                                 [            ]
                                 [   del(z)   ]
```

`dg_kill` ([*none,show*]) [Function]

The function `dg_kill` computes the Killing equations. The metric must be computed with `dg_metric` before this function is called. If called with `dis=show` then the killing equations will be shown, otherwise they can be accessed in the array `killeq`.

```
(%i1) dg_cords(mink2d);
(%o1)                                done
(%i2) dg_metric(init);
                                      2         2
                          ds2_in = del (x) - del (t)

                                     [ - 1  0 ]
                                g = [        ]
                                     [  0   1 ]

(%o2)                                done
(%i3) dg_kill(show);
                                              d     t
                          killeq     = - 2 (-- (xi ))
                                1, 1        dt

                                    d     x    d     t
                       killeq     = -- (xi ) - -- (xi )
                             1, 2   dt         dx

                                           d     x
                           killeq     = 2 (-- (xi ))
                                 2, 2      dx

(%o3)                                done
```

Appendix A Function and Variable index

D

dg_cords 2
dg_curl 14
dg_derivs 3
dg_diverg 15
dg_ffc 17
dg_grad 13
dg_jacobian 4
dg_kill 18
dg_laplac 16
dg_metric 4

C

cord_in 1
cord_ot 1
cord_tr 2
ds2_to_lg 12

G

get_ctensor_vars 12

S

set_ctensor_vars 12
show_cords 3

D

dg_minkowski 2